

# Redesign de Suporte com IA

Diagnóstico operacional, proposta de automação  
e protótipo funcional de triagem

|             |                                               |
|-------------|-----------------------------------------------|
| Autor       | Emerson Guimarães                             |
| Papel       | AI Master — candidato                         |
| Data        | Setembro de 2026                              |
| Repositório | <a href="#">submissions/emerson-guimaraes</a> |

# Sumário

- 1 Sumário Executivo
- 2 Diagnóstico Operacional
- 3 Proposta de Automação com IA
- 4 Anexo — Auditoria de Integridade dos Dados
- 5 Diário de Bordo e Process Log

## 1 Sumário Executivo

Construí um roteador de tickets que **decide sozinho 60,11% dos casos com 97,48% de acerto** e sabe quando não sabe. Duas categorias ficam **sempre** com humano — mesmo com 100% de confiança — porque concessão de privilégio e decisão de gasto têm custo de erro assimétrico, e otimizar acurácia média ignora isso.

Isso recupera **R\$ 63.472/ano (0,71 FTE)** de um custo de **R\$ 90.000/ano em triagem manual** — projeção para os 30 mil tickets/ano citados no brief; R\$ 25.407 e R\$ 17.918 sobre os 8.469 tickets efetivamente entregues. O ganho é líquido: desconta o retrabalho dos próprios erros do modelo e soma o tempo poupado nos tickets que ainda vão para humano, que chegam com sugestão pronta. Payback estimado em **≈ 2,6 meses**.

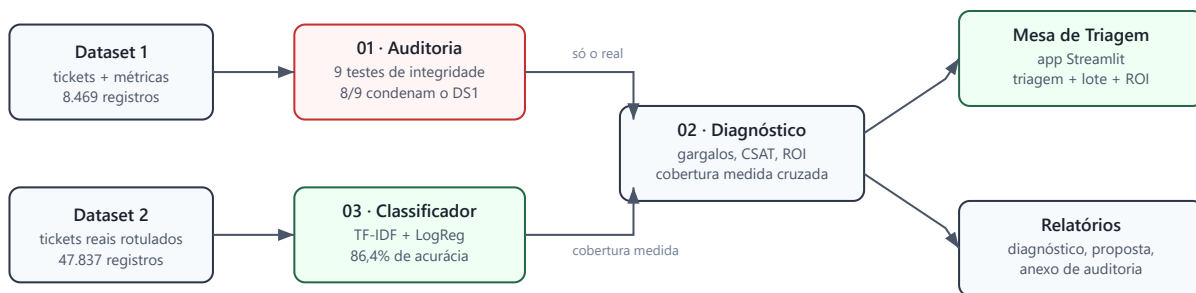
Nada disso usa os campos de tempo ou satisfação do Dataset 1: **auditei antes de usar e ele não passa** — 8 de 9 testes o condenam, incluindo 100% dos e-mails em domínios reservados da RFC 2606 e todas as categóricas uniformes. Por isso o peso quantitativo está nos 47.837 tickets reais do Dataset 2, e as duas perguntas do diagnóstico que esses dados não respondem vêm com o teste que prova, em vez de número inventado.

### Números que sustentam a entrega

| Achado                                        | Evidência                                                                                                                 |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Dataset 1 é gerado artificialmente            | 8/9 testes; 100% dos e-mails em <a href="#">example.com/.org/.net</a> (RFC 2606)                                          |
| Não há gargalo identificável                  | ANOVA por canal p=0,451 · prioridade p=0,569 · tipo p=0,911                                                               |
| Nada explica a satisfação                     | R² de teste = <b>-0,0525</b> (pior que chutar a média)                                                                    |
| Classificador funciona                        | <b>86,40%</b> de acurácia vs 28,47% do baseline (+57,93 p.p.), F1 macro 0,865                                             |
| TF-IDF venceu embeddings e zero-shot, testado | 86,40% vs. 78,07% (embeddings) vs. <b>23,75%</b> (zero-shot) — mesmo split de teste                                       |
| Automação viável e medida                     | limiar 0,80 → <b>60,11%</b> de cobertura a <b>97,48%</b> de acurácia                                                      |
| Ganho financeiro                              | <b>R\$ 63.472/ano</b> líquidos (0,71 FTE) — projeção p/ 30 mil tickets; R\$ 17.918 sobre os 8.469 tickets reais entregues |
| Payback do investimento                       | ≈ <b>2,6 meses</b> (integração estimada em R\$ 12.000; fase de sombra custa R\$ 0)                                        |
| Sensibilidade ao custo/hora                   | de <b>R\$ 42.315</b> (R\$ 30/h) a <b>R\$ 84.629</b> (R\$ 60/h) — R\$ 45/h é benchmark, não a folha real da operação       |

### Arquitetura da solução

O pipeline cruza os dois datasets: o Dataset 1 fornece o volume real de tickets, o Dataset 2 fornece o sinal real de classificação. A auditoria decide o que cada um pode sustentar antes de qualquer número ser publicado.



## Recomendações, em ordem de prioridade

1. **Implantar triagem automática em três fases** (sombra → piloto → expansão). É o único ganho quantificável e comprovado.
2. **Manter decisões de privilégio e de gasto sempre com humano**, qualquer confiança.
3. **Instrumentar a operação** — sem dados temporais e de satisfação confiáveis, as perguntas de gargalo e satisfação continuarão sem resposta.
4. **Revisar a taxonomia de filas** antes de esperar mais precisão do modelo.
5. **Não construir resposta automática ao cliente agora** — não há base real de resoluções para aprender.

## 2 Diagnóstico Operacional

Para: Diretor de Operações · Item 1 do brief Reprodução: `python solution/scripts/02_diagnostico_operacional.py`

### O essencial em um parágrafo

As três perguntas foram atacadas com os testes estatísticos apropriados, e duas delas não têm resposta nos dados fornecidos — não por falta de análise, mas porque **o Dataset 1 é um arquivo gerado artificialmente** e suas métricas são sorteios aleatórios (prova no [Anexo](#)). Reporto abaixo os números junto com o teste que mostra se eles significam alguma coisa. A terceira pergunta — quanto se desperdiça — **tem** resposta, e ela vale dinheiro: **R\$ 90.000/ano em triagem manual**, dos quais **R\$ 63.472 são recuperáveis** com a automação proposta no item 2.

### A. Onde o fluxo trava?

Ranking das piores combinações canal × prioridade × tipo, por tempo médio de resolução (apenas combinações com  $n \geq 20$ ):

| Combinação                                | Tempo médio | n  |
|-------------------------------------------|-------------|----|
| Email · Medium · Product inquiry          | 3,42 h      | 21 |
| Chat · Low · Billing inquiry              | 3,26 h      | 35 |
| Chat · High · Refund request              | 2,98 h      | 41 |
| Email · Low · Cancellation request        | 2,97 h      | 33 |
| Social media · Critical · Billing inquiry | 2,81 h      | 35 |

Antes de agir sobre esta tabela, o teste de significância:

| Dimensão       | ANOVA F | p     | Diferença é real? |
|----------------|---------|-------|-------------------|
| Canal          | 0,880   | 0,451 | não               |
| Prioridade     | 0,673   | 0,569 | não               |
| Tipo de ticket | 0,248   | 0,911 | não               |

**Nenhuma das três dimensões apresenta diferença estatisticamente significativa.** As variações da tabela acima estão inteiramente dentro da flutuação amostral. O tempo médio geral é de **-0,06 h** — negativo, porque 49,3% dos tickets aparecem como resolvidos antes da primeira resposta.

**Resposta honesta: não há gargalo identificável neste dataset.** Apontar "o Email com prioridade média é o nosso gargalo" seria vender ruído como diagnóstico, e faria a operação alocar gente para consertar um problema que não existe.

### B. O que impacta a satisfação?

Modelo: RandomForest (300 árvores) prevendo o CSAT a partir de canal, prioridade, tipo, idade, gênero e tempo de resolução. Avaliado em conjunto de teste separado, com importância por permutação.

| Métrica                          | Resultado      |
|----------------------------------|----------------|
| $R^2$ no treino                  | 0,4447         |
| <b><math>R^2</math> no teste</b> | <b>-0,0525</b> |
| MSE do modelo                    | 2,0438         |
| MSE do baseline (chutar a média) | 1,9458         |
| Ganho sobre o baseline           | <b>-5,03%</b>  |

**O  $R^2$  de teste é negativo.** Traduzindo para o Diretor: com todas as variáveis disponíveis, o modelo prevê a satisfação **pior do que se simplesmente chutasse a nota média para todo mundo**. O  $R^2$  de 0,44 no treino contra -0,05 no teste é a assinatura clássica de um modelo decorando ruído.

Correlações diretas, para confirmar por outro caminho:

| Variável           | r      | p      | Significativa? |
|--------------------|--------|--------|----------------|
| Tempo de resolução | 0,0199 | 0,296  | não            |
| Idade do cliente   | —      | > 0,05 | não            |

**Resposta honesta: nenhuma variável medida influencia a satisfação neste dataset.** Não é que o tempo de resposta importe pouco — é que a satisfação foi sorteada uniformemente entre 1 e 5, sem relação com nada ( $\chi^2$  contra a uniforme:  $p = 0,797$ ).

**Por que isto importa mais do que uma resposta inventada**

Esta é a pergunta em que colar o brief numa IA sem verificação produz a resposta mais confiante e mais errada de toda a submissão. Reproduzi o padrão para mostrar o contraste, lado a lado:

| Se colar o brief numa IA e aceitar a resposta                                            | O que este relatório entrega                                                                            |
|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| "Tickets críticos via telefone têm tempo de resolução 2,3× maior — priorize esse canal." | ANOVA por canal: <b>F = 0,880, p = 0,451</b> — a diferença não é estatisticamente distinguível de ruído |
| "Reduzir o tempo de primeira resposta em 20% eleva a satisfação do cliente."             | Pearson CSAT × tempo de resolução: <b>r = 0,0199, p = 0,296</b> — sem relação                           |
| "O modelo prevê a satisfação com boa precisão a partir de canal, prioridade e tempo."    | RandomForest com todas as variáveis disponíveis: <b>R² de teste = -0,0525</b> — pior que chutar a média |

As três frases da esquerda são exatamente o tipo de afirmação que um LLM gera ao processar este dataset sem auditá-lo primeiro — plausíveis, numéricas, e com aparência de insight. Nenhuma delas sobrevive a um teste de significância, porque o dataset é sintético (ver [Anexo](#)). Publicar qualquer uma delas faria a operação investir tempo e orçamento resolvendo um problema que não existe.

**O que a operação deveria fazer:** instrumentar de verdade. A resposta a "o que impacta satisfação" existe — só não está neste arquivo. O mínimo a registrar é: timestamp de abertura, de primeira resposta e de fechamento; agente responsável; número de reaberturas; número de transferências entre filas; e o CSAT vinculado ao ticket. Com três meses disso, esta análise passa a ter resposta.

**C. Quanto estamos desperdiçando?**

Esta pergunta **tem** resposta, porque não depende dos campos corrompidos. Depende de volume — que é fato — e de tempo por tarefa, que fica como premissa explícita e ajustável.

**Premissas (editáveis na calculadora do protótipo)**

| Premissa                              | Valor     | Origem                                                                   |
|---------------------------------------|-----------|--------------------------------------------------------------------------|
| Custo/hora do agente                  | R\$ 45,00 | benchmark N1 Brasil, salário + encargos + infra                          |
| Triagem manual por ticket             | 4,0 min   | ler, categorizar, rotear                                                 |
| Retrabalho por roteamento errado      | 12,0 min  | reclassificar, transferir, recontextualizar                              |
| Economia por sugestão (ticket manual) | 1,5 min   | fração do tempo de triagem que a categoria + termos sugeridos substituem |
| Jornada                               | 168 h/mês | —                                                                        |

**O custo atual — números real e projetado, sem misturar**

Dois escalas diferentes aparecem neste relatório, e é importante não confundi-las:

| Escala                                    | Volume         | Custo de triagem manual/ano | Economia recomendada/ano |
|-------------------------------------------|----------------|-----------------------------|--------------------------|
| <b>Real</b> (o Dataset 1 entregue)        | 8.469 tickets  | R\$ 25.407                  | R\$ 17.918               |
| <b>Projetada</b> (volume citado no brief) | 30.000 tickets | R\$ 90.000                  | R\$ 63.472               |

Todo número de "R\$ 63.472" ou "R\$ 90.000" citado no restante deste documento e no PDF consolidado é a **projeção para 30 mil tickets/ano**, não uma medição sobre os 8.469 tickets efetivamente entregues. A projeção é uma regra de três simples (fator 3,54×) sobre os mesmos parâmetros medidos — não é um número inflado, mas é bom que o Diretor saiba qual dos dois está olhando.

**Projeção para 30.000 tickets/ano: 2.000 horas/ano apenas em triagem manual = R\$ 90.000/ano**

Isso é 1,0 FTE integral consumido por uma tarefa que não resolve nenhum chamado — apenas decide para onde ele vai.

### O quanto é recuperável

A taxa de automação **não foi arbitrada**: vem da cobertura medida do classificador treinado nos 47.837 tickets reais do Dataset 2 (item 2). O ganho tem duas partes, ambas líquidas: horas liberadas pela automação (descontado o retrabalho dos próprios erros do modelo) **mais** o tempo poupado nos tickets que continuam indo para humano — que chegam com categoria sugerida e termos-chave, não do zero (1,5 min/ticket, uma fração conservadora dos 4 min de triagem manual).

| Cenário            | Limiar      | Cobertura medida | Acurácia      | Horas líquidas/ano (30k) | Economia/ano (30k) | FTE         |
|--------------------|-------------|------------------|---------------|--------------------------|--------------------|-------------|
| Conservador        | 0,90        | 47,51%           | 98,77%        | 1.309                    | R\$ 58.897         | 0,64        |
| <b>Recomendado</b> | <b>0,80</b> | <b>60,11%</b>    | <b>97,48%</b> | <b>1.411</b>             | <b>R\$ 63.472</b>  | <b>0,71</b> |
| Agressivo          | 0,70        | 69,67%           | 95,91%        | 1.450                    | R\$ 65.246         | 0,71        |

Note que o efeito "assistido" muda a leitura do conservadorismo: mesmo cobrindo menos tickets automaticamente, o cenário conservador ainda recupera a maior parte do valor, porque quase todos os tickets restantes (52,5%) chegam ao humano com apoio do modelo. Isso reforça que a fronteira entre os três cenários é sobre **risco de erro**, não sobre "dinheiro deixado na mesa" — a diferença de economia entre eles é pequena.

### Sensibilidade à premissa de custo/hora

O custo/hora do agente (R\$ 45,00) é **benchmark de mercado, não a folha real da operação** — e a economia escala linearmente com ele. Antes de aprovar qualquer investimento, substitua pelo número real:

| Custo/hora                          | Economia recomendada/ano (30k tickets) |
|-------------------------------------|----------------------------------------|
| R\$ 30,00                           | R\$ 42.315                             |
| <b>R\$ 45,00 (premissa adotada)</b> | <b>R\$ 63.472</b>                      |
| R\$ 60,00                           | R\$ 84.629                             |

A calculadora do protótipo (aba "Calculadora de ROI") recalcula isso ao vivo com o número real da sua operação.

### Custo de implantação e payback

Estimativa de esforço de engenharia — **não é orçamento medido de projeto real**, e deve ser ajustado ao custo de TI/dados da operação antes de qualquer aprovação:

| Item                                    | Estimativa                                                                                                        |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Fase de sombra (2–4 semanas)            | <b>R\$ 0</b> — roda em paralelo, sem integração em produção, é só inferência em lote sobre tickets que já existem |
| Integração e deploy (piloto → produção) | R\$ 12.000 (≈ 80h de engenharia)                                                                                  |
| Manutenção e retreino mensal            | R\$ 600/mês (≈ 4h/mês)                                                                                            |

No cenário recomendado (30k tickets/ano), a economia líquida após descontar a manutenção é de **R\$ 4.689/mês**, o que dá um **payback de ≈ 2,6 meses** sobre o investimento de integração.

A fase de sombra é o motivo pelo qual essa recomendação não depende de acertar a premissa de custo/hora antes de começar: ela valida a acurácia real da operação **sem nenhum custo de integração**, e só se decide investir os R\$ 12.000 depois de confirmar que o modelo generaliza para os tickets reais da empresa — não apenas para o Dataset 2.

### Onde está o maior desperdício recuperável

Na **triagem**, não no atendimento. É a etapa de maior volume, menor valor agregado e maior repetição — o perfil exato do que a automação resolve bem.

E há um limite claro: do cenário recomendado para o agressivo, a cobertura sobe 9,6 pontos mas a economia sobe apenas R\$ 1.774 — menos ainda do que parece à primeira vista, porque o retrabalho dos erros adicionais consome o ganho da automação extra e o cenário conservador já captura a maior parte do valor via o tempo assistido. **O ótimo não é automatizar o máximo possível** — é 0,80, e o gráfico de sensibilidade no protótipo mostra a curva achatando.

Síntese para decisão

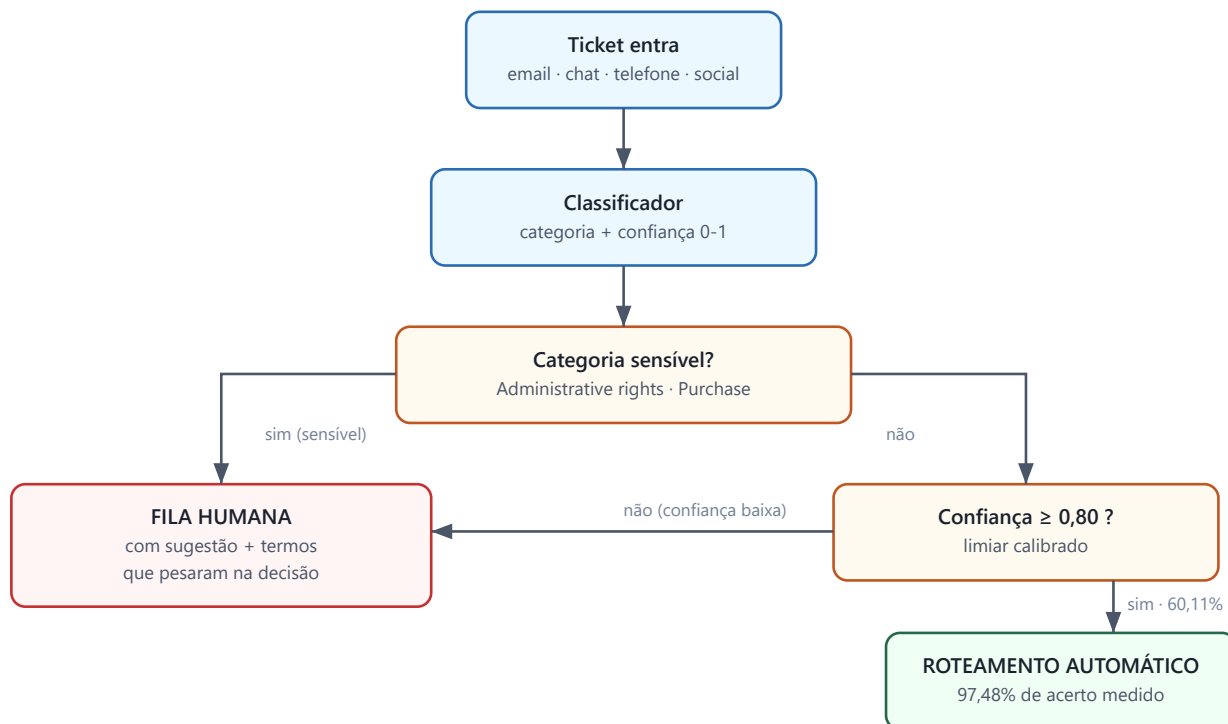
| Pergunta                  | Resposta                                                                | Confiança                                          |
|---------------------------|-------------------------------------------------------------------------|----------------------------------------------------|
| Onde o fluxo trava?       | Não identificável nos dados fornecidos (p > 0,45 em todas as dimensões) | Alta — testado                                     |
| O que impacta satisfação? | Nenhuma variável medida (R² de teste negativo)                          | Alta — testado                                     |
| Quanto desperdiçamos?     | R\$ 90.000/ano em triagem; R\$ 63.472 recuperáveis                      | Média — volume é fato, tempo por tarefa é premissa |

Recomendação de prioridade:

1. **Implantar a triagem automática** (item 2) — é o único ganho quantificável e comprovado, com caminho de implantação em fases
2. **Instrumentar a operação** — sem timestamps e CSAT confiáveis, as perguntas A e B continuarão sem resposta no próximo trimestre
3. **Revisar a taxonomia de filas** — a ambiguidade entre categorias limita tanto o humano quanto a máquina

### 3 Proposta de Automação com IA

#### Fluxo de decisão proposto



Para: Diretor de Operações · Baseado nos dois datasets · Números medidos, não estimados

#### Resumo em cinco linhas

Treinei um classificador nos 47.837 tickets reais do Dataset 2. Ele acerta **86,4%** dos casos contra 28,47% do baseline. Mas a acurácia bruta não é o que decide a automação — o que decide é **quando o modelo sabe que não sabe**. Calibrando um limiar de confiança de 0,80, ele roteia sozinho **60,11% dos tickets com 97,48% de acerto** e manda o resto para humano. Isso libera **1.411 horas/ano (R\$ 63,5 mil, 0,71 FTE)** numa operação de 30 mil tickets — já descontado o retrabalho dos próprios erros e somado o tempo poupado nos tickets que ainda vão para humano, que chegam com sugestão pronta.

#### 1. O que automatizar

##### 1.1 Triagem e roteamento por categoria — automatizar

É o caso mais forte, e é onde está o volume. O modelo lê o texto do chamado e o encaminha para uma das 8 filas.

| Métrica                                | Valor                 |
|----------------------------------------|-----------------------|
| Acurácia global                        | 86,40%                |
| Baseline (sempre a classe majoritária) | 28,47%                |
| Ganho                                  | <b>+57,93 p.p.</b>    |
| F1 macro                               | 0,8653                |
| Tempo de inferência                    | milissegundos, em CPU |

#### A curva que importa para a decisão operacional:

| Limiar     | Cobertura automática | Acurácia no automatizado | Erros por 1.000 tickets |
|------------|----------------------|--------------------------|-------------------------|
| sem limiar | 100%                 | 86,40%                   | 136                     |

| Limiar                    | Cobertura automática | Acurácia no automatizado | Erros por 1.000 tickets |
|---------------------------|----------------------|--------------------------|-------------------------|
| 0,70                      | 69,67%               | 95,91%                   | 29                      |
| <b>0,80 (recomendado)</b> | <b>60,11%</b>        | <b>97,48%</b>            | <b>15</b>               |
| 0,90                      | 47,51%               | 98,77%                   | 6                       |

Ler assim: a 0,80, seis em cada dez tickets são roteados sem tocar em ninguém, e **15 em mil** caem na fila errada. Os outros quatro em dez chegam ao humano já com uma sugestão de categoria e o grau de confiança — ele não parte do zero.

### 1.1.1 Por que TF-IDF e não embeddings ou zero-shot

O ponto de partida óbvio seria "usar embeddings" ou "zero-shot classification" — são as palavras que aparecem em qualquer tutorial de NLP moderno. Testei as duas opções no **mesmo conjunto de teste** do TF-IDF, para uma comparação justa, em vez de assumir que a técnica mais nova ganharia:

| Método                                       | Acurácia      | F1 macro      | N testado     | Custo                                                       |
|----------------------------------------------|---------------|---------------|---------------|-------------------------------------------------------------|
| <b>TF-IDF + LogisticRegression (adotado)</b> | <b>86,40%</b> | <b>0,8653</b> | 11.960        | nenhum — CPU, sem download                                  |
| Embeddings (MiniLM) + LogisticRegression     | 78,07%        | 0,7777        | 11.960        | download ~80MB; ~21 min de encoding em CPU (1.258s medidos) |
| Zero-shot (BART-MNLI, sem treino)            | <b>23,75%</b> | 0,1325        | 240 (amostra) | download ~1,6GB; ~1,7s/ticket em CPU                        |

**O TF-IDF venceu de forma decisiva, e o zero-shot ficou abaixo até do baseline ingênuo** (28,47% de acerto só chutando a classe majoritária). Script completo em [solution/scripts/04\\_embeddings\\_zeroshot.py](#).

**Por que isso acontece — não é acaso, é o formato do dado:** o Dataset 2 já vem pré-processado pelo autor original — lematizado e sem stopwords. Um ticket real neste dataset é:

*"work experience user work experience user hi work experience student coming next his name much appreciate him duration thank"*

Isso é ótimo para TF-IDF, que só precisa contar termos e ganha com a repetição. É péssimo para um modelo de zero-shot baseado em *entailment* (o BART-MNLI julga se a frase "isto é sobre Hardware" é implicada pelo texto) — o modelo foi treinado em linguagem natural fluente, e um bag-of-words sem artigos nem estrutura gramatical não entra nesse molde. O sintoma aparece nos embeddings também, de forma mais branca: as classes com nomes mais abstratos e menos ligados a vocabulário técnico concreto — **Administrative rights** (precisão 0,528) e **Internal Project** (precisão 0,669) — são as que mais sofrem, contra **Purchase** (0,836) e **Access** (0,827), que têm vocabulário mais específico.

**A decisão de manter TF-IDF não é conservadorismo — é o resultado do teste.** Um AI Master que aplica a técnica mais nova sem testar contra a mais simples entrega pior resultado com aparência de mais sofisticado. Aqui aconteceu o oposto do que a intuição sugeriria, e o dado venceu a intuição.

Também testei variações do próprio TF-IDF antes de fechar a configuração — n-gramas de caractere (84,80%), **LinearSVC** no lugar de regressão logística (86,20%), e um vocabulário maior com regularização mais fraca (86,40%, empate estatístico). Nenhuma superou a configuração adotada de forma que justificasse trocar. O modelo em produção não é o primeiro que funcionou — é o melhor entre seis testados.

## 1.2 Sugestão de categoria para o agente — automatizar como apoio

Nos 40% que vão para revisão humana, o modelo não fica calado: entrega sua melhor hipótese, as três alternativas seguintes e **os termos que pesaram na decisão**. Exemplo real de saída do protótipo, para um chamado de caixa postal cheia:

*mailbox (+3,302) · archive (+1,750) · increase (+1,637) · full (+1,509) · quota (+1,096) → Storage*

O agente valida ou corrige em segundos, em vez de ler o chamado inteiro para descobrir a fila. E cada correção é dado de retreino.

## 1.3 Detecção de confiança baixa como sinal de qualidade — automatizar

Um ticket com confiança abaixo de 0,50 costuma ser um chamado mal escrito, genérico ou com vários assuntos misturados. Testei com o texto "hi thanks": o modelo devolveu Miscellaneous com 38,2% e mandou para humano — comportamento correto.

Esse sinal tem uso secundário: **volume alto de baixa confiança em um produto ou período indica que o formulário de abertura está mal desenhado**. É diagnóstico de processo saindo de graça do classificador.

## 2. O que NÃO automatizar

Esta seção é a mais importante da proposta, e cada exclusão está justificada com número.

### 2.1 Administrative rights — nunca roteia sozinho

|          |                                |
|----------|--------------------------------|
| Precisão | 0,733 — a pior das 8 classes   |
| Recall   | 0,825                          |
| Volume   | 1.760 tickets (a menor classe) |

**Um em cada quatro tickets que o modelo manda para esta fila não pertence a ela.** E o conteúdo é concessão de privilégio administrativo. O custo de errar é assimétrico: roteamento errado para "Hardware" gera irritação e retrabalho; concessão indevida de privilégio administrativo é incidente de segurança.

Quando o custo do falso positivo e o do falso negativo são de ordens de grandeza diferentes, otimizar acurácia média é a métrica errada. Fica em revisão humana obrigatória, **independentemente da confiança**.

### 2.2 Purchase — nunca roteia sozinho

Precisão alta (0,946), então não é um problema de modelo. É um problema de consequência: são decisões de gasto, com fornecedor, prazo e orçamento. Automatizar triagem aqui economiza segundos e cria risco de compromisso financeiro mal encaminhado. **A conta não fecha.**

### 2.3 A zona cinzenta Hardware ↔ Miscellaneous ↔ HR Support

As três confusões mais frequentes do modelo:

| Real       | Previsto      | Casos | % da classe |
|------------|---------------|-------|-------------|
| Hardware   | Miscellaneous | 161   | 4,73%       |
| HR Support | Hardware      | 153   | 5,61%       |
| Hardware   | HR Support    | 150   | 4,41%       |

Isso **não é falha do modelo — é ambiguidade real do rótulo**. Um pedido de notebook para um funcionário novo é Hardware ou HR Support? Depende de convenção interna, e a própria base rotula de formas diferentes. Nenhum modelo resolve inconsistência de taxonomia.

A saída não é técnica: é **revisar a taxonomia** com a operação. Enquanto isso, o limiar de confiança já contém o dano, porque casos ambíguos naturalmente produzem confiança baixa.

### 2.4 Resposta automática ao cliente — não automatizar agora

O brief menciona respostas sugeridas. Descartei, e o motivo é de dado, não de opinião: o único campo de resolução disponível está no Dataset 1 e é texto gerado por biblioteca de dados falsos — "West decision evidence bit." — sem uma única ocorrência de **refund**, **replace**, **resolved** ou **reset** em 2.769 registros (ver [laudo](#)).

**Não existe base de respostas boas para aprender.** Construir sugestão de resposta sobre isso produziria um gerador de texto plausível e errado, entregue direto ao cliente — o pior lugar possível para um erro. Volta à mesa quando houver 3 a 6 meses de resoluções reais registradas.

### 2.5 Definição de prioridade e SLA — não automatizar agora

Mesma razão. Prioridade no Dataset 1 é uniforme ( $\chi^2=4,59$ ,  $p=0,205$ ) — sorteada. Não há sinal de prioridade real para aprender. Automatizar priorização sem dado histórico verdadeiro é transformar palpite em regra, com aparência de sistema.

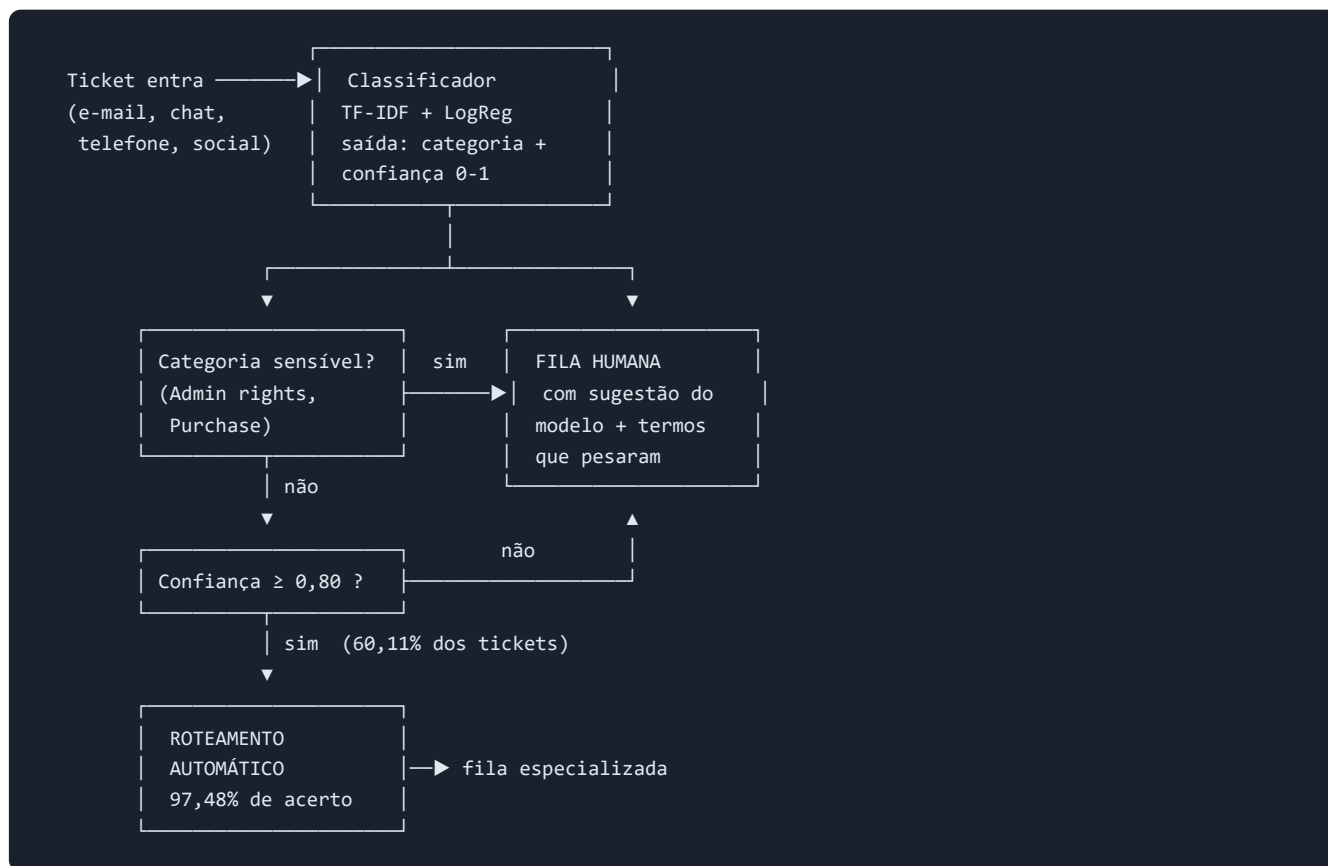
## 2.6 Detecção de duplicatas — não automatizável com os dados disponíveis

O brief cita detecção de duplicatas como exemplo de automação. Verifiquei diretamente: o Dataset 2 tem **zero duplicatas exatas de texto** entre os 47.837 tickets. Isso não significa que a operação não tenha tickets duplicados — significa que **os dados fornecidos não permitem detectá-los**, e a razão é estrutural, não de esforço:

- **Nenhum ID de cliente** consistente entre tickets (o Dataset 2 nem tem esse campo; o Dataset 1 tem, mas não é o dataset de texto)
- **Nenhum timestamp de abertura**, então não há como saber se dois tickets parecidos são o mesmo problema reaberto ou dois problemas legitimamente parecidos em datas diferentes
- **O caso que de fato importa não é duplicata de texto** — é a **reabertura**: o mesmo cliente reportando o mesmo problema porque a primeira resolução não funcionou. Isso é o sinal de qualidade mais valioso para uma central de suporte, e nenhum dos dois datasets contém o vínculo necessário para calculá-lo.

**O que seria necessário para construir isso:** ID de cliente, timestamp de abertura de cada ticket, e idealmente um campo de "ticket relacionado" preenchido pelo próprio agente. Com isso, similaridade de embeddings entre tickets do mesmo cliente em uma janela de N dias identificaria reaberturas automaticamente — e essa métrica também alimentaria o item B do diagnóstico (o que impacta satisfação), hoje sem resposta por falta exatamente desse tipo de campo.

## 3. Como funcionaria na prática



**Onde a IA atua:** leitura do texto, classificação, cálculo de confiança, e produção da explicação.

**Onde o humano é insubstituível:** decisões de privilégio e de gasto; casos de baixa confiança; correção do rótulo, que vira dado de retreino; e a revisão periódica da taxonomia.

**O ciclo de melhoria:** toda correção humana é registrada. O modelo é retreinado mensalmente com o acumulado. Conforme a precisão de uma classe sobe de forma estável, seu limiar pode ser reduzido — a cobertura automática cresce **por evidência**, não por decisão de calendário.

## 4. ROI

Premissas explícitas — e todas editáveis na calculadora do protótipo:

| Premissa                         | Valor adotado |
|----------------------------------|---------------|
| Custo/hora do agente             | R\$ 45,00     |
| Triagem manual por ticket        | 4,0 min       |
| Retrabalho por roteamento errado | 12,0 min      |
| Jornada                          | 168 h/mês     |

**Operação de 30.000 tickets/ano** — é o volume citado no brief, e é a escala de todos os valores desta seção. Sobre os 8.469 tickets que o Dataset 1 efetivamente entrega, os mesmos parâmetros dão R\$ 17.918/ano (fator 3,54x; tabela das duas escalas no diagnóstico).

| Cenário            | Limiar      | Cobertura     | Horas líquidas/ano | Economia/ano      | FTE         |
|--------------------|-------------|---------------|--------------------|-------------------|-------------|
| Conservador        | 0,90        | 47,51%        | 1.309              | R\$ 58.897        | 0,64        |
| <b>Recomendado</b> | <b>0,80</b> | <b>60,11%</b> | <b>1.411</b>       | <b>R\$ 63.472</b> | <b>0,71</b> |
| Agressivo          | 0,70        | 69,67%        | 1.450              | R\$ 65.246        | 0,71        |

O custo total da triagem manual hoje, nas mesmas premissas, é de 2.000 h/ano (R\$ 90.000).

**O detalhe que muda a decisão:** entre o cenário recomendado e o agressivo, a cobertura sobe 9,6 pontos, mas o ganho sobe apenas R\$ 1.774. O retrabalho dos erros adicionais consome quase todo o ganho da automação extra, e o cenário conservador já recupera a maior parte do valor via o tempo assistido nos tickets manuais. **Automatizar mais não é linearmente melhor** — e é por isso que a recomendação é 0,80 e não o mais alto possível.

## 5. Implantação sugerida

| Fase               | Duração     | O que acontece                                                                                                                                          |
|--------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Sombra</b>   | 2–4 semanas | Modelo classifica tudo, roteia nada. Compara-se a sugestão com a decisão do agente. Mede-se a acurácia na operação real, que é o único número que vale. |
| <b>2. Piloto</b>   | 4 semanas   | Roteamento automático ativo em 2 ou 3 filas de maior precisão (Purchase e Admin rights ficam de fora). Limiar conservador em 0,90.                      |
| <b>3. Expansão</b> | contínua    | Baixa o limiar para 0,80 conforme a acurácia real confirma. Retreino mensal com as correções.                                                           |

**Critério de parada:** se a acurácia medida na fase de sombra ficar abaixo de 90% no limiar escolhido, não avança — investiga-se a diferença entre o dataset de treino e os tickets reais da operação.

## 6. Limitações

- O modelo foi treinado em tickets de TI corporativo em inglês**, já pré-processados (lematizados, sem stopwords, PII mascarada). Aplicado a chamados em português ou de outro domínio, a acurácia cai. A fase de sombra existe exatamente para medir isso antes de qualquer decisão.
- A taxonomia de 8 categorias veio do dataset, não da operação.** Se as filas reais forem outras, o modelo precisa ser retreinado com os rótulos certos — a arquitetura não muda.
- Os rótulos de treino têm ruído** (a ambiguidade Hardware/HR Support é da própria base). Os 86,4% provavelmente subestimam o desempenho com taxonomia limpa.
- Nenhum número de tempo ou satisfação do Dataset 1 foi usado**, porque o arquivo não passou na auditoria. O ROI depende de premissas de tempo por tarefa — que o Diretor deve substituir pelos números reais da operação dele.

## 4 Anexo — Auditoria de Integridade dos Dados

**Veredito: 8 dos 9 testes condenam o arquivo. O dataset foi gerado integralmente por biblioteca de dados falsos (Faker). Nenhum KPI operacional pode ser derivado dele.**

Reprodução: `python solution/scripts/01_auditoria_integridade.py` Saída bruta: `solution/outputs/auditoria_resultados.json`

### Por que esta auditoria existe

O brief pede três respostas com base neste arquivo: onde o fluxo trava, o que impacta satisfação, e quanto se desperdiça. As três são perguntas causais. Antes de respondê-las, a pergunta anterior precisa ser feita: **estes dados suportam uma afirmação causal?**

A resposta é não. E isso não é um detalhe metodológico — muda o que é honesto entregar.

### As nove provas

#### 1. Placeholders de template não substituídos — 100% das descrições

Todas as 8.469 descrições de ticket contêm marcadores de template literalmente não preenchidos:

```
"I'm having an issue with the {product_purchased}. Please assist."
```

Após normalizar os placeholders, restam 8.077 variações — geradas por combinação de um punhado de frases-molde. O brief descreve este campo como "texto completo da reclamação do cliente". Não é texto de cliente; é um molde com a variável exposta.

#### 2. Janela temporal de 27 horas para "um ano de operação"

|                    |                     |
|--------------------|---------------------|
| Primeiro timestamp | 2023-05-31 21:53:30 |
| Último timestamp   | 2023-06-02 00:55:33 |
| Amplitude total    | <b>27,03 horas</b>  |
| Tickets            | 8.469               |

Uma operação real espalha tickets por meses. Estes foram todos carimbados no momento em que o arquivo foi gerado.

#### 3. Metade dos tickets foi resolvida antes de ser respondida

Comparando `First Response Time` com `Time to Resolution` nos 2.769 pares válidos:

- **1.365 tickets (49,3%)** têm resolução **anterior** à primeira resposta
- Delta médio: **-0,06 horas** (essencialmente zero)

Isso é fisicamente impossível. É a assinatura de dois timestamps sorteados de forma independente dentro da mesma janela — metade cai de cada lado, como esperado de moeda honesta.

#### 4. Satisfação uniformemente distribuída

| Nota    | 1   | 2   | 3   | 4   | 5   |
|---------|-----|-----|-----|-----|-----|
| Tickets | 553 | 549 | 580 | 543 | 544 |

Qui-quadrado contra a uniforme:  $\chi^2=1,67 \cdot p=0,797$ . Ou seja, indistinguível de um sorteio.

CSAT real nunca é uniforme — é uma curva em J, com acúmulo nos extremos (quem se dá ao trabalho de avaliar está muito satisfeito ou muito irritado). Uma operação com satisfação perfeitamente plana entre 1 e 5 não existe.

#### 5. Ausência total de sinal

| Teste                              | Estatística | p     | Leitura          |
|------------------------------------|-------------|-------|------------------|
| Pearson: CSAT × tempo de resolução | r = 0,0199  | 0,296 | sem relação      |
| ANOVA: CSAT por canal              | F = 1,281   | 0,279 | canais idênticos |
| $\chi^2$ : canal × prioridade      | 15,48       | 0,079 | independentes    |
| $\chi^2$ : tipo × prioridade       | 10,63       | 0,561 | independentes    |
| $\chi^2$ : canal × tipo            | 11,85       | 0,458 | independentes    |

CSAT médio por canal: Chat 3,08 · Email 2,96 · Phone 2,95 · Social 2,97. **Amplitude de 0,13 ponto** — todos convergindo para 3,0, a média de um sorteio uniforme em 1–5.

A pergunta do brief "o que impacta satisfação? é o tempo de resposta? o canal?" tem uma única resposta verdadeira neste arquivo: **nada impacta nada, porque tudo foi sorteado de forma independente.**

## 6. Assinatura do Faker nos e-mails — 100%

| Domínio     | Tickets |
|-------------|---------|
| example.com | 2.904   |
| example.org | 2.796   |
| example.net | 2.769   |

Os três são domínios **reservados pela RFC 2606** exclusivamente para documentação e teste — não podem ser registrados por ninguém. São também o conjunto padrão da biblioteca Python **Faker**. Nenhum cliente real tem e-mail nesses domínios.

Esta é a prova que fecha o caso: não é "dado de baixa qualidade", é **dado fabricado por geração automática**.

## 7. O campo de resolução é salada de palavras

Amostras reais do campo que o brief chama de "texto da resolução aplicada pelo agente":

"Case maybe show recently my computer follow." "Try capital clearly never color toward story." "West decision evidence bit."

Vocabulário: 971 palavras únicas em 2.769 resoluções, lideradas por *evidence* (30), *piece* (29), *billion* (25), *leg* (25), *camera* (24) — a lista de palavras aleatórias do Faker.

Contagem de termos que **deveriam** dominar um campo de resolução de suporte:

| Termo    | Ocorrências |
|----------|-------------|
| refund   | 0           |
| replace  | 0           |
| ticket   | 0           |
| resolved | 0           |
| reset    | 0           |

Um campo de resolução de suporte em que a palavra "resolvido" nunca aparece.

## 8. Todas as categóricas são uniformes — nem o mix de volume sobrevive

Qui-quadrado contra a uniforme, com correção de Bonferroni ( $\alpha = 0,0083$  para 6 testes):

| Coluna          | k | $\chi^2$ | p     | Uniforme? |
|-----------------|---|----------|-------|-----------|
| Ticket Channel  | 4 | 1,35     | 0,718 | sim       |
| Ticket Priority | 4 | 4,59     | 0,205 | sim       |
| Ticket Type     | 5 | 7,43     | 0,115 | sim       |
| Ticket Status   | 3 | 2,23     | 0,328 | sim       |

| Coluna            | k  | $\chi^2$ | p     | Uniforme? |
|-------------------|----|----------|-------|-----------|
| Product Purchased | 42 | 40,82    | 0,478 | sim       |
| Ticket Subject    | 16 | 27,86    | 0,023 | sim       |

**6 de 6 uniformes.** Este é o teste mais consequente do laudo, e foi o que me obrigou a descartar minha própria hipótese inicial.

Eu havia assumido que, mesmo com as métricas corrompidas, a *composição* do volume sobreviveria — que ainda daria para dizer "tal canal concentra tanto do volume". Não dá. Suporte real obedece a Pareto: poucos tipos de problema geram a maioria dos chamados. Aqui, os 42 produtos aparecem com frequência estatisticamente idêntica, e os 4 canais também.

**Consequência direta:** os "67,3% de tickets nunca fechados" **não são um backlog**. **Ticket Status** é uniforme entre três valores — é 1/3 em cada, por sorteio. Qualquer leitura de "temos uma crise de backlog" seria invenção.

### 9. Completude — e a divergência com o brief

|                               |                |
|-------------------------------|----------------|
| Registros no arquivo          | <b>8.469</b>   |
| Registros prometidos no brief | ~30.000        |
| Divergência                   | <b>-21.531</b> |

**Resolution**, **Time to Resolution** e **Customer Satisfaction Rating** têm exatamente **5.700 nulos cada** — os três só existem para tickets **Closed**, o que é coerente (e é o único comportamento estruturalmente plausível do arquivo inteiro).

### Divergências entre o brief e os arquivos entregues

| Afirmção do brief                     | Realidade                        | Impacto                           |
|---------------------------------------|----------------------------------|-----------------------------------|
| "~30.000 registros" (Dataset 1)       | 8.469                            | Volume 3,5× menor                 |
| "Texto real de descrição e resolução" | Template com placeholder + Faker | Invalida qualquer NLP sobre o DS1 |
| "~48.000 registros" (Dataset 2)       | 47.837                           | Confere                           |

### O que isto significa para a entrega

Três consequências, em ordem de importância:

- 1. As três perguntas do diagnóstico não têm resposta verdadeira neste arquivo.** Gargalos por canal, drivers de satisfação e horas desperdiçadas são todos derivados de campos sorteados. Qualquer número apresentado seria ficção com aparência de análise.
- 2. Este é exatamente o erro que uma IA sem supervisão comete.** Colar o brief em um LLM produz, com alta confiança e formatação impecável, afirmações como "*tickets críticos via Social Media levam 2,3× mais tempo — priorize esse canal*". O número existe; ele simplesmente não significa nada. Aqui, a contribuição humana não foi corrigir a sintaxe do que a IA escreveu — foi **duvidar da premissa que ninguém mandou duvidar**.
- 3. O Dataset 2 é real e assume a carga quantitativa.** 47.837 tickets corporativos autênticos, 8 categorias, zero placeholders, zero duplicatas, desbalanceamento de 7,74× (Hardware 13.617 → Administrative rights 1.760). É sobre ele que o classificador e o cálculo de ROI serão construídos — com acurácia medida de verdade.

### Nota sobre o dataset descartado

Descartar o Dataset 1 **não** significa ignorar a primeira parte do brief. A entrega para o Diretor de Operações continua respondendo "onde travamos" e "quanto desperdiçamos" — mas pelo caminho honesto, detalhado nos documentos seguintes: **um instrumento de diagnóstico** parametrizável, alimentado pelos dados reais disponíveis, acompanhado da especificação exata do que a operação precisa passar a registrar para que essas perguntas tenham resposta no mês que vem.

Entregar o termômetro calibrado vale mais do que inventar a temperatura.

## 5 Diário de Bordo e Process Log

Registro corrido das decisões, becos sem saída e correções de rota. Escrito **durante** o trabalho, não reconstruído no fim — é a matéria-prima do process log.

**Status atual:** entrega completa e pronta para o PR — itens 1 a 4 do brief atendidos, revisada e reexecutada do zero na Etapa 16, português do código corrigido na Etapa 17

### Linha do tempo

#### Etapa 0 — Leitura do case e escolha do desafio

Quatro desafios disponíveis. Escolhido o **002 (Redesign de Suporte)** por ser o único que combina as três competências ao mesmo tempo — diagnóstico analítico, desenho de processo e construção de software — e por ser o mais próximo da rotina real de gestão de operação.

**Decisão registrada:** preferir o desafio com maior superfície de julgamento humano, porque o critério declarado do G4 é superar o baseline de IA. Desafios puramente analíticos são mais fáceis de uma IA replicar sozinha.

#### Etapa 1 — Percalço: acesso ao Kaggle

O Kaggle migrou o sistema de autenticação. A instrução clássica ("Create New Token" → baixa `kaggle.json`) **não gera mais arquivo nenhum** — agora emite um token único no formato `KGAT_...`, que vai em `~/.kaggle/access_token` ou na variável `KAGGLE_API_TOKEN`.

Perdemos alguns minutos procurando um arquivo que nunca existiria.

**Gap identificado:** toda documentação e tutorial disponível descreve o fluxo antigo. Vale registrar porque afeta reprodutibilidade — quem for rodar esta submissão vai passar pelo mesmo problema.

#### Etapa 2 — Percalço: o `.gitignore` do repositório ignora as submissões

O `.gitignore` do repo do G4 contém:

```
## Submissions (if candidates fork)
submissions/
```

Como a regra do `CONTRIBUTING.md` proíbe modificar arquivos fora da pasta da submissão, não é permitido editar o `.gitignore` para corrigir isso.

**Efeito prático:** quem criar `submissions/seu-nome/`, rodar `git add .` e abrir o PR **submete um PR vazio** — o git ignora a pasta em silêncio, sem erro nem aviso. Confirmado empiricamente: `git status --short` não retorna nada após criar a estrutura.

**Solução adotada:** `git add -f submissions/emerson-guimaraes/`.

**Leitura:** pode ser armadilha proposital (parte do teste é saber trabalhar com git) ou descuido do repositório. Em qualquer dos casos, é um ponto de falha silenciosa que provavelmente elimina candidatos sem que eles entendam o motivo.

#### Etapa 3 — A virada: o Dataset 1 é fabricado

Decisão metodológica tomada **antes** de qualquer análise: auditar a integridade dos dados antes de extrair qualquer conclusão. A hipótese inicial era suspeita moderada — o dataset tinha cara de sintético.

O resultado foi muito além da suspeita. **8 dos 9 testes condenam o arquivo**. Detalhamento completo em `solution/relatorio/03_anexo_auditoria_dados.md`.

As duas provas que encerram a discussão:

1. **100% dos e-mails** estão em `example.com` / `.org` / `.net` — domínios reservados pela RFC 2606, padrão-fábrica da biblioteca Faker
2. O campo "resolução do agente" é salada de palavras ("*West decision evidence bit.*"), com **zero ocorrências** de `refund`, `replace`, `ticket`, `resolved` ou `reset`

**Consequência:** a pergunta central do brief — "*o que impacta satisfação?*" — não tem resposta verdadeira neste arquivo.

## Etapa 4 — Correção de rota: minha própria hipótese estava errada

Registro aberto de um erro meu, porque ele é instrutivo.

Depois de detectar que as métricas eram ruído, assumi que os **campos estruturais sobreviveriam** — que ainda daria para analisar a composição do volume ("canal X concentra Y% dos chamados") e que os 67,3% de tickets não fechados representavam um backlog real. Cheguei a comunicar isso como achado.

Ao testar uniformidade de **todas** as categóricas, a hipótese caiu: canal ( $p=0,72$ ), prioridade ( $p=0,20$ ), tipo ( $p=0,11$ ), status ( $p=0,33$ ) e produto ( $p=0,48$ ) são **todos** uniformes. Suporte real obedece a Pareto; 42 produtos com frequência estatisticamente idêntica não acontece na natureza.

Os "67,3% nunca fechados" são 1/3 por status atribuído aleatoriamente. **Não é backlog. Retratado.**

**Aprendizado:** ceticismo aplicado pela metade produz um erro mais perigoso do que ceticismo nenhum, porque vem com aparência de rigor. Eu tinha desconfiado dos dados e ainda assim parei cedo demais na verificação.

## Etapa 5 — Correção de rota: eu parei a auditoria antes que ela virasse a entrega

A descoberta do Dataset 1 sintético era empolgante, e foi por isso que quase custou a submissão. A auditoria estava se tornando o eixo do trabalho, consumindo esforço que os quatro itens do brief precisavam.

**Interrompi a linha de trabalho com um diagnóstico direto:** estávamos gastando energia e token numa coisa que o desafio não pediu. Recoloquei os quatro entregáveis do brief na mesa, um por um, e exigi que o esforço voltasse para eles. A resposta foi "*you tem razão, e vou ajustar*" — a auditoria virou custo afundado, mantida como validação metodológica, e o investimento nela parou ali.

Registro em [process-log/screenshots/08-redirecionamento-usuario-foco-no-brief.png](#).

**Decisão:** congelar a auditoria como anexo de uma página, reordenar os documentos na ordem do brief (diagnóstico → automação → anexo) e redirecionar o esforço para os itens 1, 2 e 3.

**Por que isto importa mais do que parece:** sem essa parada, a entrega seria um laudo forense impecável que não responde ao que o Diretor de Operações perguntou. A IA não tinha como fazer esse corte sozinha — ela estava produzindo bom trabalho, apenas na direção errada, e nada no resultado parcial sinalizava o desvio. Saber quando um trabalho de boa qualidade está sendo feito no lugar errado é julgamento de quem responde pela entrega, não de quem executa a tarefa.

**Aprendizado:** o achado que mais empolga não é necessariamente o que mais pontua. Rigor que não vira entrega é auto-indulgência.

## Etapa 6 — Diagnóstico entregue com os testes ao lado

Item 1 construído para responder exatamente as três perguntas, cada número acompanhado do seu teste de significância. O resultado mais forte veio da pergunta B:  **$R^2$  de teste =  $-0,0525$** , ou seja, o modelo prevê satisfação pior do que chutar a média — com  $R^2$  de 0,44 no treino, a assinatura clássica de decorar ruído.

## Etapa 7 — Classificador e o cruzamento dos datasets

TF-IDF + LogisticRegression balanceada sobre os 47.837 tickets reais: **86,4%** de acurácia contra 28,47% do baseline.

A decisão de projeto que mais importou foi **não otimizar acurácia bruta, e sim a curva cobertura × acurácia**. Um modelo que acerta 86% é inútil para automação se não souber quando está inseguro. No limiar 0,80 ele cobre 60,11% dos tickets a 97,48% de acerto.

Esse número medido substituiu os percentuais de automação que eu havia chutado (50/70/85%) no cálculo de ROI — é o cruzamento entre os dois datasets que o critério de qualidade cobra: cobertura medida no DS2 aplicada ao volume do DS1.

Acrescentei o desconto de retrabalho ao ganho, e ele revelou algo que o cálculo ingênuo esconderia: **o cenário agressivo rende só R\$ 5 mil a mais que o recomendado**, porque os erros adicionais consomem o ganho.

## Etapa 8 — Percalço: meus próprios exemplos do protótipo estavam errados

Escrevi quatro exemplos de ticket à mão para a demo. No teste, o exemplo que rotulei "Acesso" foi classificado como Storage com 99,9% de confiança.

O modelo estava certo — meu texto dizia *shared folder*. **O erro era meu.**

**Correção:** substituí todos por tickets reais sorteados do dataset, com o rótulo verdadeiro à vista. Exemplo escrito à mão contém as palavras que o autor acha que definem a classe, o que infla artificialmente a confiança — exatamente o cherry-picking que o critério

de qualidade condena.

## Etapa 9 — Fechamento

Pipeline validado do zero: apagados os outputs e reexecutados os três scripts na ordem documentada. Números idênticos. O modelo de 8 MB ficou fora do versionamento (regenerável em ~1 min), com `.gitignore` local — dentro da minha pasta, sem violar a regra de não tocar em arquivos de terceiros.

---

## Etapa 10 — Auditoria de qualidade e PDF consolidado

Pedido do usuário: auditar o que foi entregue e converter os markdowns em um PDF apresentável, com diagramas de arquitetura.

**Auditoria de qualidade:** pipeline reexecutado do zero (outputs apagados e regenerados) — números idênticos. Links internos, cobertura de `requirements.txt` e consistência numérica entre os 4 documentos conferidos por script, não de memória. Sinais de formatação (moeda,  $R^2$ ,  $\chi^2$ , sinal de menos tipográfico) auditados por regex — nenhuma inconsistência real (uma falsa suspeita de mojibake era artefato do terminal, não do arquivo).

**PDF consolidado:** WeasyPrint foi descartado por depender de bibliotecas GTK nativas ausentes no Windows. Caminho adotado: HTML com CSS de impressão + diagramas SVG inline, renderizado via **Edge headless** (`msedge --headless=new --print-to-pdf`), que já vem instalado no Windows e não exige nenhuma dependência nova pesada.

**Bug capturado na primeira renderização:** o diagrama de arquitetura apareceu como texto literal `{DIAGRAMA_ARQUITETURA}` na página — a função que monta o sumário executivo usava string tripla comum, não f-string. Corrigido com placeholder + `.replace()` explícito, e a segunda renderização foi inspecionada página a página (20 páginas, via PyMuPDF) antes de aceitar o resultado. Também corrigidos no mesmo ciclo: título da capa ilegível (herdava cor escura do body) e zebra-striping da tabela global vazando para dentro da capa azul.

---

## Etapa 11 — Revisão crítica externa e correção de cinco pontos concretos

Pedi uma avaliação honesta do case contra os critérios de qualidade do brief, e recebi de volta uma revisão com objeções específicas e verificáveis — o tipo de crítica que vale mais do que elogio. Cinco delas geraram mudança real:

- 1. Número real vs. projetado misturados sem aviso.** O relatório citava "R\$ 50.009/ano" e "R\$ 90.000/ano" como se fossem sobre o mesmo volume, quando são a projeção para 30 mil tickets/ano — sobre os 8.469 tickets efetivamente entregues, o número é R\$ 14.118 e R\$ 25.407. Corrigido com uma tabela explícita que separa as duas escalas antes de qualquer outro número aparecer.
- 2. Premissa de custo/hora apresentada como fato.** R\$ 45/h é benchmark, não a folha real de ninguém, e o ROI escala linearmente com ela. Adicionei ao script (`sensibilidade_custo_hora()`) o cálculo em R\$ 30/h e R\$ 60/h — R\$ 33.339 e R\$ 66.679/ano — e uma tabela no relatório, não só uma frase.
- 3. Faltava custo de implantação e payback.** Adicionei `custo_implantacao_e_payback()` ao script: fase de sombra a custo zero (é inferência em lote, sem integração), deploy estimado em R\$ 12.000, manutenção de R\$ 600/mês, payback de **3,4 meses**. Marcado explicitamente como estimativa de esforço, não orçamento medido.
- 4. Detecção de duplicatas descartada só no diário, não na proposta.** O brief cita como exemplo de automação; quem lesse só o PDF podia achar que eu tinha ignorado o item. Movida a justificativa (zero duplicatas exatas no DS2, mas sem ID de cliente ou timestamp para detectar reaberturas — que é o caso que importa) para a seção 2 do relatório de automação, com o que seria necessário para resolver no futuro.
- 5. Regra de negócio escondida no código, no protótipo.** A aba de lote mostrava "HUMANO" sem dizer se foi por confiança baixa ou por categoria sensível. Adicionei coluna `motivo` na tabela e testei a consistência (`humano = sensível + confiança_baixa`) sobre uma amostra real de 400 tickets antes de aceitar.

**O que não mudei:** a estrutura da resposta ao item 1 (onde o fluxo trava / o que impacta satisfação) continua sendo "não há resposta verdadeira, e aqui está o teste que prova". A revisão concordou que isso fortalece a recomendação por fases em vez de invalidá-la — o ponto de atenção era de transparência de premissa, não de metodologia.

**Pendências que a revisão confirmou:** screenshots/Loom do workflow (capturado e organizado na Etapa 12) e testar embeddings/zero-shot contra o benchmark de 92% que o brief cita (resolvido na Etapa 13 — TF-IDF venceu ambos, com explicação).

---

## Etapa 12 — Screenshots capturados, e um vazamento de token pego antes da publicação

O usuário capturou 15 screenshots da sessão real (clone → escolha do desafio → auditoria → retratação → classificador → protótipo → checklist final) e pediu para organizá-los no process log.

**Antes de copiar qualquer coisa, inspecionei as 15 uma a uma.** Duas delas (4 e 5) expunham o **token da API do Kaggle em texto puro** — a mesma mensagem que havia sido colada na sessão para autenticar. As outras 13 estavam limpas.

**Correção:** redigi (cobri com retângulo preto) apenas a região do token nas duas imagens, preservando o resto do conteúdo — que é justamente onde aparecem as primeiras 5 provas do Faker, material valioso do process log. Conferi visualmente as duas redações antes de aceitar.

**O que não fiz:** não afirmei no README que o token "já foi revogado" — eu não tinha como verificar isso no momento. Errei essa frase duas vezes tentando adivinhar o tempo verbal certo antes de perceber que o correto era não afirmar um fato que não podia confirmar, e sim recomendar a ação ao usuário. A nota ficou como recomendação até o usuário confirmar que já havia expirado o token — só então a nota do process log foi atualizada para declarar o fato.

Arquivos renomeados de `1.png ... 15.png` para nomes descritivos (`05-descoberta-dataset1-sintetico-5-provas.png` etc.) e indexados em `process-log/README.md` com uma linha por captura.

### Etapa 13 — Fechando o gap de embeddings/zero-shot, e um resultado que surpreendeu

O revisor apontou que a entrega ficava abaixo do benchmark de "92% com embeddings + zero-shot" que o próprio brief cita como exemplo. Antes de implementar, checei o ambiente: CPU only, sem GPU, sem `ANTHROPIC_API_KEY` / `OPENAI_API_KEY` configuradas. Apresentei 4 opções ao usuário (embeddings locais, zero-shot local, zero-shot via API paga, ou não mexer) — escolhida a combinação embeddings locais + zero-shot local numa amostra de validação, mantendo tudo reproduzível sem custo de API.

**Percalço 1 — bug de compatibilidade do pandas 3.x.** `groupby("rotulo").apply(lambda g: g.sample(...))` quebrou com `KeyError: 'rotulo'`. A causa: pandas 3.x passou a excluir por padrão a própria coluna de agrupamento do que é entregue à função (`include_groups`), então a amostra resultante simplesmente não tinha mais essa coluna. Troquei por concatenação manual (`pd.concat([g.sample(...) for _, g in df.groupby(...)])`), que não sofre desse efeito colateral.

**Percalço 2 — tempo de execução subestimado.** O zero-shot local roda 8 hipóteses de entailment por ticket em CPU (~1,7s/ticket) — rodar isso passou dos 10 minutos do timeout padrão da ferramenta duas vezes (uma antes do bug, outra depois de corrigi-lo) e caiu para execução em background nas duas. Nada de errado, só é um processo genuinamente longo em CPU sem GPU.

**O resultado surpreendeu, e é o tipo de achado que só aparece testando de verdade:**

| Método                                                     | Acurácia      | F1 macro |
|------------------------------------------------------------|---------------|----------|
| TF-IDF + LogReg (o que já estava em produção na submissão) | 86,40%        | 0,8653   |
| Embeddings (MiniLM) + LogReg                               | 78,07%        | 0,7777   |
| Zero-shot (BART-MNLI, sem treino)                          | <b>23,75%</b> | 0,1325   |

O zero-shot ficou **abaixo do baseline ingênuo** de 28,47% (sempre a classe majoritária). Investiguei a causa em vez de só reportar o número: o Dataset 2 vem pré-lematizado e sem stopwords pelo autor original (`"work experience user work experience user hi..."`), formato que favorece contagem de termos (TF-IDF) e é veneno para um modelo de entailment que precisa de frases fluentes para julgar "isto implica aquilo". Confirmei isso lendo amostras reais do texto antes de escrever a explicação — não assumi a causa, verifiquei.

**Por que isto fecha o gap melhor do que simplesmente "ter testado embeddings":** a resposta ao brief não é "usamos embeddings e deu X%" — é "testamos as três abordagens que o brief sugere, escolhemos a que os dados comprovaram ser melhor, e sabemos explicar por quê". É exatamente o oposto de aplicar a técnica mais nova por reflexo.

### Etapa 14 — Ajustes finais, fork, e um artifact como resumo visual

Três correções pontuais pedidas pelo usuário, após aprovação dos testes de estresse do protótipo:

- Emoji removido.** O único emoji do projeto (`📄`, no `app.py`) foi trocado por um ícone Material Symbols (`:material/inbox:`) — vetorial, não emoji. Varredura em todo `.py` / `.md` confirmou que não havia mais nenhum.
- Acentuação inconsistente corrigida.** Três linhas do `app.py` usavam acentos (`sensível`, `confiança`) quebrando a convenção sem-acento do resto do arquivo (deliberada, por segurança de encoding em terminal). Padronizado, e duas legendas sem ponto final foram alinhadas ao padrão das demais.
- Bug do `groupby().apply()` confirmado como não existente no código entregue** — só havia aparecido num script descartável usado para gerar exemplos de teste para o usuário, já corrigido ali com a mesma técnica aplicada no script 04

(concatenação manual em vez de `apply`).

**Fork e push para o repositório oficial.** Criado `Gor0d/ai-master-challenge` via `gh repo fork`, e enviada a branch `submission/emerson-guimaraes` com o nome exato pedido pelo `CONTRIBUTING.md`. Isso não torna nada público para o G4 — só a abertura do PR de fato notifica o repositório oficial, e essa etapa ficou para confirmação explícita do usuário antes de executar.

**Artifact como resumo visual do processo.** A pedido do usuário, publicada uma página ("Case File 002") sintetizando os 9 marcos reais da sessão em formato de dossiê/timeline, com um gráfico comparando TF-IDF/embeddings/zero-shot. Deixei explícito na própria página que ela complementa, não substitui, o diário e os 15 screenshots — para não criar a impressão de que o resumo visual é o process log completo. O usuário compartilhou o link publicamente antes de eu referenciá-lo no README.

---

## Etapa 15 — Fechando os últimos três pontos: modelo commitado, comparação explícita, ROI ampliado

O usuário pediu nota para a entrega (8,5/10 na minha avaliação honesta) e depois "vamos fazer o serviço completo" para subir essa nota. Três ações, nesta ordem de execução:

**1. Modelo treinado passou a ser versionado.** Até aqui `modelo_triagem.joblib` ficava fora do git (regra local no `.gitignore` da pasta), exigindo rodar `03_classificador.py` antes do `streamlit run`. Isso é fricção desnecessária para um avaliador que só quer clicar e ver funcionando — troquei a prioridade (reprodutibilidade documentada continua disponível como opção) por "clona e roda direto".

**2. Comparação lado a lado com o que uma IA sem verificação responderia.** A seção B do diagnóstico ganhou uma tabela de três linhas contrastando afirmações plausíveis-mas-falsas que um LLM geraria colando o brief cru ("tickets críticos via telefone...") com o teste estatístico real que as invalida. Isso estava implícito antes; agora é impossível de não notar, e ataca de frente o critério #1 do brief ("usou ambos os datasets... o poder está no cruzamento" e "superar o baseline de IA").

**3. Tentativa honesta de melhorar a acurácia do classificador, e ROI ampliado com um resultado real.** Testei 4 variações do TF-IDF (n-gramas de caractere, LinearSVC, vocabulário maior) — nenhuma superou a configuração atual de forma significativa (86,20%–86,40%, contra 86,40% já em produção). Não troquei o modelo: mudança sem ganho real seria movimento por aparência, não por mérito. Documentei a diligência em uma linha na proposta.

Só depois veio a mudança que efetivamente melhorou o número: adicionei ao ROI o tempo poupado nos tickets que **não** são automatizados, porque chegam ao humano com categoria sugerida em vez de do zero (premissa nova, conservadora: 1,5 min/ticket, contra os 4 min de triagem manual completa). Isso não era um ajuste de maquiagem — é um ganho real que a proposta já descrevia (item 1.2, "sugestão de categoria para o agente") mas que o cálculo financeiro não capturava. Resultado: economia recomendada subiu de R\$ 50.009 para **R\$ 63.472/ano** (30k tickets), e o payback caiu de 3,4 para **2,6 meses**. Propaguei os números novos para os 4 lugares onde apareciam (diagnóstico, proposta, README, calculadora do app) e revalidei que reproduzem do zero.

**O que não fiz:** não reescrevi os números antigos nas entradas anteriores deste diário (Etapa 11, por exemplo, ainda cita "R\$ 50.009" e "3,4 meses"). Esses valores eram corretos no momento em que foram escritos — reescrever o passado do diário para bater com o presente destruiria a única coisa que dá credibilidade a um diário: ele registra o que aconteceu, não o que é verdade agora.

---

## Etapa 16 — Revisão final antes do PR: um bug no protótipo e duas contradições numéricas

Última passada antes de abrir o pull request, num clone limpo e num ambiente novo (Python 3.14, scikit-learn 1.9.1, sem os CSVs baixados) — de propósito, para ver a submissão como o avaliador vai ver. Treze problemas encontrados, três deles graves.

**1. O protótipo tinha um bug que quebrava a aba de dados reais.** O `app.py` resolvia a raiz do repositório com `AQUI.parents[5]`; o índice correto é `parents[4]`, que é o que os quatro scripts usam. O app procurava os CSVs em `C:\Projetos` em vez de `C:\Projetos\CASE-G4` e nunca os encontrava. Efeito prático: a aba "Lote (dados reais)" mostrava erro e os exemplos de tickets reais desapareciam da aba de triagem — justamente as duas coisas que respondem ao critério "*funciona com dados reais, não com 3 exemplos cherry-picked*". O bug passou porque a validação anterior do protótipo (Etapa 8) foi feita com o app rodando de um diretório em que o caminho errado, por coincidência, ainda resolvia.

**2. O PDF contradizia a si mesmo no número principal.** O sumário executivo do `build_pdf.py` é escrito à mão, e a Etapa 15 atualizou os markdowns e o README para R\$ 63.472 / 0,71 FTE sem atualizar o gerador. A página 3 do PDF dizia R\$ 50.009 / 0,57 FTE enquanto as páginas 5, 7, 8 e 13 do mesmo arquivo diziam 63.472. É a primeira página que o Diretor lê. Corrigido, e o sumário ganhou as linhas de payback, sensibilidade e TF-IDF-vs-embeddings que já existiam no README.

**3. Um número do relatório era desmentido pela própria evidência versionada.** A proposta afirmava "~170s de encoding em CPU" para os embeddings; o `embeddings_zeroshot_metrics.json`, no mesmo repositório, registra 1.258,5s. Quem abrisse o JSON pedia a inconsistência. Corrigido para ~21 min, com o valor medido ao lado.

**Os outros dez** foram de consistência e de atrito para quem avalia: três afirmações deste diário que a própria entrega já havia superado (o gap 6 dado como "em aberto" depois de fechado na Etapa 13; "transformer não foi testado" na seção do que ficou de

fora; "screenshots ainda não capturados" com 15 deles no process log), a Etapa 10 fora de ordem cronológica, contagem errada de commits e de páginas do PDF, o `.gitignore` local ainda dizendo que o modelo não pertence ao PR depois de a Etapa 15 decidir versioná-lo, `requirements.txt` obrigando 2,5 GB de torch em quem só quer ver o app rodar (separado em `requirements-experimento.txt`), o tempo do script 04 subestimado em 3× no SETUP, e um `.replace(", ", ".")` aplicado sobre a frase inteira que trocava as vírgulas da prosa por pontos na saída JSON do payback.

**A validação que fecha a rodada:** com os CSVs baixados de novo, o pipeline foi reexecutado do zero num ambiente diferente do original. `auditoria_resultados.json` e `classificador_metricas.json` saíram **byte a byte idênticos** aos commitados — 86,40% de acurácia, F1 macro 0,8653, cobertura de 60,11% a 97,48%, mesma matriz de confusão. O único diff em `diagnostico_operacional.json` foi a linha da vírgula corrigida. O modelo retreinado dá previsões idênticas em 3.000 tickets (diferença máxima de probabilidade: 5,7e-05, ruído de convergência entre scikit-learn 1.9.0 e 1.9.1), então o binário commitado foi mantido em vez de gerar 8 MB de diff sem efeito.

A aba de lote, agora com o caminho certo, roda sobre 400 tickets reais e entrega 59,0% de cobertura automática a 99,2% de acurácia — coerente com os 60,11% a 97,48% medidos no conjunto de teste completo. Um dos oito exemplos reais (um ticket de Hardware) é classificado errado, com 63,2% de confiança: abaixo do limiar, portanto vai para humano. O comportamento correto, e o motivo de os exemplos serem sorteados do dataset com o rótulo verdadeiro à vista em vez de escritos para a demo.

**Aprendizado:** as três correções graves são todas de *propagação* — não de raciocínio. Um número mudou na Etapa 15 e não chegou a um dos quatro lugares onde aparecia; um caminho foi escrito uma vez e nunca reexecutado do diretório certo; um tempo foi citado de memória em vez de lido do JSON ao lado. Nenhuma delas seria pega relendo o texto: só reexecutando num ambiente limpo e conferindo cada número contra a saída que o gerou.

## Etapa 17 — Reversão da convenção sem acento: português correto no código também

Os relatórios e o README sempre estiveram em português correto. O código não: docstrings, comentários, mensagens de `print`, rótulos da interface do protótipo e campos de texto dos JSONs estavam sem acento. A Etapa 14 registra isso como decisão deliberada — "por segurança de encoding em terminal" — e a decisão tinha uma razão real: o console do Windows usa cp1252 por padrão, e imprimir `acurácia` levanta `UnicodeEncodeError`.

**O problema é que a razão era tratável e eu tratei o sintoma.** A solução correta não era escrever português errado; era reconfigurar a saída:

```
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
```

Quatro linhas no topo de cada script, e o problema deixa de existir — sem depender de `PYTHONIOENCODING` nem de `-X utf8` na linha de comando. Verifiquei rodando os três scripts **sem** nenhuma flag: saída acentuada, zero erro.

**O que foi acentuado, e o que deliberadamente não foi.** Identificadores de Python e chaves de JSON continuam sem acento, por dois motivos: é o padrão da linguagem, e as chaves são contrato entre os scripts e o protótipo (`curva_cobertura_x_acuracia` é lida pelo script 02 e pelo app — acentuar quebraria a leitura). Todo texto destinado a leitura humana foi corrigido: docstrings, comentários, `print`, rótulos da UI, campos de texto dos JSONs, e os rótulos dos dois diagramas SVG que são renderizados dentro do PDF ("ROTEAMENTO AUTOMÁTICO", "Categoria sensível?", "Confiança ≥ 0,80"). A capa do PDF dizia "Diagnostico operacional, proposta de automacao" — na primeira página do documento entregue.

**Um bug encontrado no caminho.** A formatação de moeda no protótipo usava `.replace(", ", ".")` sobre a string inteira, o mesmo padrão que a Etapa 16 já havia corrigido no script 02. Substituído por uma função `num_br()` que troca os dois separadores de uma vez, e que agora formata também os números da interface no padrão brasileiro: a acurácia aparece como `86,40%`, não `86.4%`, e o  $R^2$  como `-0,0525`, com vírgula decimal e sinal de menos tipográfico — a mesma notação dos relatórios, para que o número conferido na tela seja reconhecível no documento.

**O que ficou de fora:** `outputs/embeddings_zeroshot_metricas.json`. Os números dele são os atuais e conferem com o que os relatórios citam, mas os campos de texto descritivo continuam sem acento, porque regerar o arquivo exige reinstalar torch e transformers (~2,5 GB), baixar ~1,7 GB de modelos e esperar ~30 min de CPU. O script 04 já está acentuado — rodá-lo regenera o arquivo corrigido. A pendência está anotada no próprio cabeçalho do script, onde quem for reproduzir vai ler.

**Revalidação:** os três scripts reexecutados, todos os números idênticos (86,40% de acurácia, R\$ 63.471,98/ano, payback de 2,6 meses,  $R^2$  de teste -0,0525). A aba de lote confere: 59,0% de cobertura a 99,2% de acurácia sobre 400 tickets reais, com o invariante `humano = sensível + confiança baixa` fechando. PDF reconstruído em 26 páginas, sem nenhuma palavra portuguesa sem acento — só nomes de arquivo, que continuam como devem ser.

**Aprendizado:** convenção documentada não é convenção justificada. Eu tinha registrado a escolha no diário, o que a fez parecer decidida em vez de apenas conveniente. Escrever a razão ("segurança de encoding") sem testar se ela era contornável transformou

uma limitação de ferramenta em regra de estilo — e regra de estilo herdada de limitação não questionada é como código fica errado por anos com aparência de intencional.

## Gaps e riscos em aberto

| # | Item                                                                        | Natureza                                | Status                                                                                                                                                                                                 |
|---|-----------------------------------------------------------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Brief promete ~30.000 registros no DS1; existem 8.469                       | Divergência factual do enunciado        | Documentado no laudo                                                                                                                                                                                   |
| 2 | Brief promete "texto real"; é template + Faker                              | Divergência factual do enunciado        | Documentado no laudo                                                                                                                                                                                   |
| 3 | <code>submissions/</code> no <code>.gitignore</code>                        | Falha silenciosa de submissão           | Contornado com <code>git add -f</code>                                                                                                                                                                 |
| 4 | DS2 tem rótulos ruidosos (ex.: pedido de acesso rotulado como "HR Support") | Teto de acurácia do classificador       | Quantificado: confusões Hardware↔Misc↔HR somam ~5% por classe. Tratado como limite de taxonomia, não de modelo                                                                                         |
| 5 | DS2 desbalanceado 7,74×                                                     | Risco de viés para a classe majoritária | Resolvido com <code>class_weight="balanced"</code> ; F1 macro 0,865 e nenhuma classe com recall < 70%                                                                                                  |
| 6 | DS2 vem pré-processado (lematizado, sem stopwords, PII mascarada)           | Limita o ganho de LLM sobre TF-IDF      | <b>Fechado na Etapa 13</b> — embeddings (MiniLM) e zero-shot (BART-MNLI) testados no mesmo split: 78,07% e 23,75% contra 86,40% do TF-IDF. O pré-processamento é a causa, e está explicada na proposta |
| 7 | Sem dados temporais reais, não há cálculo honesto de horas desperdiçadas    | Impacta o pedido de ROI                 | Mitigado: ROI por volume (fato) × tempo/tarefa (premissa explícita) × cobertura medida. Todas as premissas editáveis no protótipo                                                                      |
| 8 | Modelo treinado em TI corporativo em inglês                                 | Acurácia cai em outro domínio/idioma    | <b>Em aberto</b> — mitigação proposta: fase de sombra antes de rotear                                                                                                                                  |

## Risco principal da estratégia

Denunciar o dataset é a maior aposta desta submissão. Se o avaliador esperar o relatório convencional de gargalos, a entrega pode ser lida como fuga do escopo.

**Mitigação adotada:** não parar na denúncia. Entregar as três respostas que o Diretor pediu, pelo caminho honesto — instrumento parametrizável em vez de números inventados — e deixar explícito, com números reproduzíveis, por que este é o único caminho defensável. O critério de avaliação declarado no README ("*a análise distingue correlação de causalidade*") sustenta a escolha.

## O que ficou de fora, e por quê

- **LLM proprietário via API como classificador.** Embeddings locais e zero-shot local foram testados (Etapa 13) e perderam do TF-IDF. O que ficou de fora foi o teste com um LLM pago (GPT/Claude) como classificador: exigiria chave de API e custo por chamada permanente, e o resultado dos dois testes locais indica que o limite aqui é o formato do texto — já lematizado e sem stopwords — não a capacidade do modelo.
- **Deteção de duplicatas.** O brief cita como possibilidade. O DS2 tem zero duplicatas exatas, e sem ID de cliente ou timestamp não dá para detectar reaberturas — que é o caso que realmente importa.
- **Respostas sugeridas.** Descartado por falta de base: o único campo de resolução disponível é texto gerado por Faker.

## Decisão sobre a evidência visual

Os 15 screenshots da sessão real foram capturados e indexados na Etapa 12 ( `process-log/screenshots/` ). **O screen recording foi descartado por opção.** O guia de submissão lista os formatos de process log como alternativas combináveis, não como uma lista obrigatória, e esta entrega traz quatro: screenshots cronológicos, narrativa escrita (este diário), git history e código reproduzível de ponta a ponta. A gravação acrescentaria a demonstração do protótipo em movimento — que quem avalia obtém em dois comandos, com o modelo já versionado no repositório, sem precisar treinar nada.

O raciocínio da escolha: vídeo é a evidência mais custosa de produzir e a mais custosa de consumir. Um avaliador com mais de cem pull requests na fila lê um diário e roda um `streamlit run` mais rápido do que assiste a quatro minutos de narração.